

PIBT on DotBots: Simulation-to-Hardware Gap in Swarm Intralogistics

Corentin Rasda

ESIROI / Inria AIO

France

rasdacorentin@gmail.com

1 Introduction

Swarm robotics trades a few heavily instrumented robots for many small, cheap units that coordinate in a decentralized way, so that system throughput scales with the size of the fleet rather than with per-unit capability. Intralogistics — automated warehouses and parcel-sorting floors of the Kiva type, where mobile robots shuttle goods between storage and picking stations — is its flagship application and the one we target here [?][?].

The coordination kernel of such a fleet is Multi-Agent Path Finding (MAPF): routing N agents from their start cells to N distinct goal cells on a shared graph without vertex or swap collisions [?]. Optimal MAPF is NP-hard [1], so deployed systems favour fast, scalable sub-optimal solvers. PIBT (Priority Inheritance with Backtracking) is one such solver [1]: it plans a single synchronous step at a time, guarantees that every agent eventually reaches its goal on biconnected graphs, and runs in predictable, near-real-time computation — properties that make it a natural fit for on-board, lifelong operation rather than offline batch planning.

The DotBot is an open-source, low-cost wheeled micro-robot. It is localized by a single Lighthouse v2 (LH2) base station and networked through MARI, a TSCH-over-BLE mesh protocol, giving sparse external positioning and constrained, contention-prone communication — exactly the regime a real deployment must tolerate [2].

We validate PIBT for DotBot warehouse-style coordination at two levels on a shared occupancy axis $\rho = N/\text{cells}$: $L0$, an abstract-grid simulation that fixes the algorithmic ceiling, and $L1$, the same planner driving real DotBots. Joining the two on ρ quantifies the **plan-to-real gap**. Our thesis is that the simulated ceiling does not survive contact with hardware: the simulation predicts cheap, near-linear coordination up to an efficiency knee at $\rho \approx 0.60$, yet real per-instance reliability already collapses at $\rho \approx 0.30$, roughly half the simulated value.

2 PIBT Algorithm

PIBT solves MAPF iteratively by prioritized planning in a unit-length time window [1]. At each timestep every agent carries a unique priority and the agents commit to their next cell in decreasing priority order, each choosing the neighbouring cell closest to its goal that no higher-priority agent has reserved. When a high-priority agent is blocked by a lower-priority one, the blocker temporarily *inherits* the higher priority and is forced to give way; if it then finds no valid move, it *backtracks* and the caller tries its next candidate. On any graph where every pair of adjacent nodes lies on a simple cycle — an obstacle-free rectangular grid is biconnected and qualifies — this scheme guarantees *reachability*: every agent



Figure 1: Real DotBot testbed: a 2000 × 2000 mm grid arena with low-cost DotBots and a single Lighthouse v2 anchor providing external localization.

reaches its goal within finite time, although not necessarily all at the same timestep.

One planning step costs $O(|A|(\Delta(G) + \log |A|))$ and stays within a few milliseconds even for hundreds of agents, so the per-step run-time is predictable. Because every decision is a local interaction between neighbouring agents, PIBT is readily decentralized and extends to lifelong pickup-and-delivery (MAPD) [1], the formulation closest to a working warehouse.

3 Results

Setup. The same PIBT planner drives both levels. The physical arena is 2000 × 2000 mm with a single LH2 anchor. $L1$ sweeps a 5 × 5 grid (25 cells, 400 mm) and adds one 8 × 8 control (64 cells, 250 mm). DotBots are commanded one synchronous PIBT step at a time through a barrier: the planner advances one step, the bots are sent to their next cells, and the loop waits until every bot is within a 75 mm arrival threshold (tightened from 100 mm) or an 8 s per-step timeout elapses, re-synchronising each bot’s cell from LH2 before the next step. Commands go through the PyDotBot stack [?] over MARI/LH2 [2]. We place one DotBot per cell and draw starts and goals uniformly at random per seed. The hardware logs hold **8 configurations** ($N = 4$ –10 on the 5 × 5 grid plus $N = 4$ on the 8 × 8 grid) over **67 runs** total.

Evaluation. For each run `l1_metrics.py` logs makespan (steps), wall-time (s), step-timeouts (bot × step events that miss the 8 s barrier), overshoot (mean/max stop distance to the target cell, mm) and throughput (goals / min). We separate two notions of success: *per-agent* = arrived/ N , and *per-instance* = all N arrived. Only the per-instance notion is comparable to $L0$ and to Okumura’s reachability criterion, so we report it as the headline metric.

L0 — the algorithmic ceiling. On the 5 × 5 grid (30 seeds, 100-step cap), per-agent success stays high across the entire sweep,

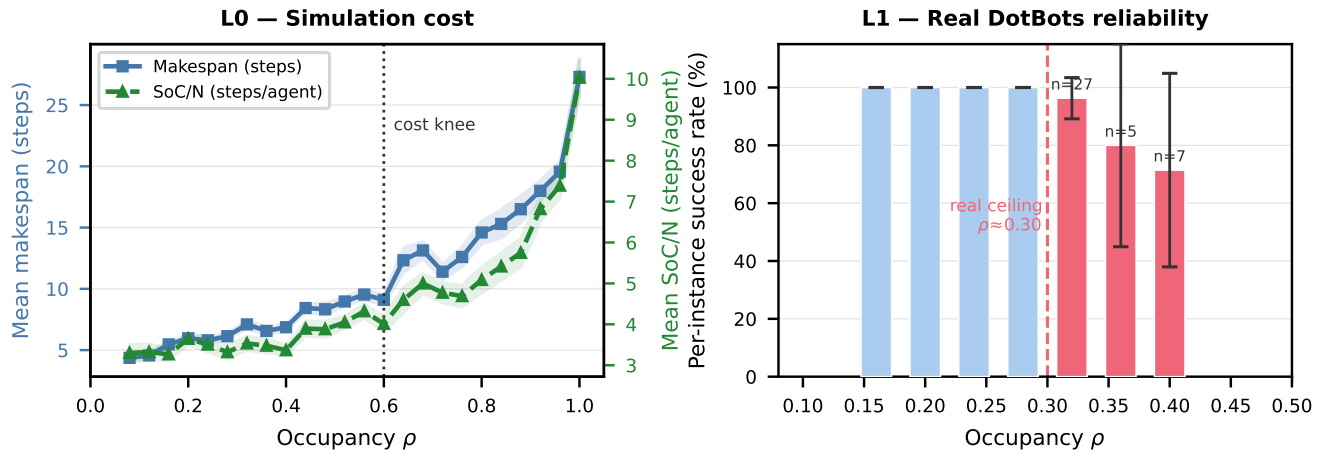


Figure 2: Plan-to-real gap on the 5×5 grid. Left (L0, simulation): mean makespan and sum-of-costs per agent (SoC/N) stay near-linear up to the cost knee at $\rho \approx 0.60$, then accelerate — the algorithmic ceiling. Right (L1, real DotBots): per-instance success holds 100% up to $\rho = 0.28$ (blue) then collapses to 96/80/71% at $\rho = 0.32/0.36/0.40$ (red), past the reliability ceiling at $\rho \approx 0.30$. The real fleet breaks down at roughly half the occupancy the simulation still deems efficient.

from 98.3% at $\rho = 0.08$ to 92.1% at $\rho = 1.00$: reachability is guaranteed and only the conservative step cap clips it. The informative ceiling is therefore in *cost*, not *success*. Both sum-of-costs and mean makespan remain near-linear up to $\rho = 0.60$ ($N = 15$: makespan 9.1 steps, sum-of-costs 60.2, detour ratio 1.46) and then accelerate sharply ($\rho = 0.80$: makespan 14.6, sum-of-costs 101.7; $\rho = 1.00$: makespan 27.3, sum-of-costs 250.7). This $\rho \approx 0.60$ knee is the algorithmic upper bound — the densest packing at which the planner still keeps per-robot cost proportional to travel distance.

L1 — real DotBots. Per-instance success is 100% for $N = 4-7$ ($\rho \leq 0.28$): all 25 runs delivered every agent, with makespans of 5.2–7.8 steps, 16–35 s wall-time, 69–90 mm mean overshoot and ≤ 2 step-timeouts per run. It then degrades monotonically with density: 96.3% at $N = 8$ ($\rho = 0.32$, 26/27 runs), 80% at $N = 9$ ($\rho = 0.36$, 4/5) and 71% at $N = 10$ ($\rho = 0.40$, 5/7). The degradation is physical, not algorithmic, and three mechanisms compound. (i) *Footprint versus cell size*: a DotBot needs room for a half-turn to reach the next waypoint; in tight cells it overshoots the 75 mm threshold (mean overshoot stays 56–90 mm, with maxima above 600 mm), the step barrier times out (timeouts climb from ≈ 2 to 5 to 7 per run over $N = 8 \rightarrow 10$, wall-time from 43 to 90 s), and the next replan sends it back — trapping it in a two-cell oscillation. This is exactly the signature observed on a dense $8 \times 8 / 250$ mm run, where only 3/6 robots reached their goals. (ii) *Timeout cascade*: because PIBT advances as one synchronous step, a single bot that misses the barrier desynchronises its neighbours’ reserved cells and breaks the local coherence PIBT depends on, so one initial timeout can fail several agents at once. (iii) *Residual rotations*: our planner forbids 2-agent swaps but not general 3+-agent rotations (no PIBT*), which on hardware surface as the very oscillations we observe. The net effect is a real per-instance reliability ceiling at $\rho \approx 0.30$ — about half the simulated cost knee at $\rho \approx 0.60$ — with throughput peaking

near 24 goals / min at $N = 6$ and falling to ≈ 10 goals / min once $N = 10$ pushes the fleet past that ceiling.

4 Conclusion & Future Work

DotBots driven by PIBT do perform warehouse-style multi-agent coordination, but the densest occupancy that is *reliable* on hardware is roughly half of what the simulation’s cost curve would permit ($\rho \approx 0.30$ versus $\rho \approx 0.60$). This is a different question from Okumura et al.’s robot demonstration [1], which established *feasibility* on the Toio platform using a position-encoded mat and point-to-point BLE. We instead characterise *reliability* under sparse external localization (a single LH2 anchor) and constrained mesh communication (MARI/TSCH) [2], and we pinpoint where it breaks. Future work moves from one-shot MAPF to lifelong Multi-Agent Pickup and Delivery (MAPD), the warehouse-native formulation [?]; runs MAPD in a decentralized fashion over the 6TiSCH mesh; and validates at scale with our industrial partner, La Poste.

References

- [1] K. Okumura, M. Machida, X. Défago, and Y. Tamura. Priority inheritance with backtracking for iterative multi-agent path finding. *Artificial Intelligence*, 310:103752, 2023.
- [2] F. Alvarado et al. DotBot: An open-source platform for swarm robotics research. 2024.